

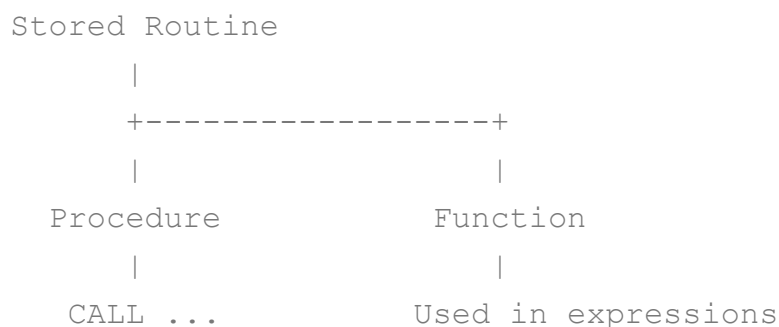
1. What is a Stored Routine?

A **stored routine** is a program stored inside the MySQL database.

There are two types:

1. **Stored Procedure**
2. **Stored Function**

Conceptually:



For example, instead of repeatedly writing:

```
SELECT name, salary
FROM employees
WHERE employee_id = 101;
```

you can store that logic in a procedure:

```
CALL GetEmployee(101);
```

The database server stores the routine and executes its SQL statements when invoked.

2. Why Use Procedures and Functions?

Stored routines are useful when:

1. **You want to reuse SQL logic**

Instead of writing the same SQL repeatedly:

```
CALL CalculateSalary(101);  
CALL CalculateSalary(102);  
CALL CalculateSalary(103);
```

2. You want centralized business logic

For example:

```
Application  
  |  
  v  
Stored Procedure  
  |  
+---- Check employee  
+---- Calculate salary  
+---- Update salary  
+---- Record transaction
```

3. Security

You can give a user permission to execute a procedure without giving them direct access to all underlying tables.

For example:

```
GRANT EXECUTE ON PROCEDURE company.PayEmployee TO 'user'@'localhost';
```

4. Reduce communication between application and database

Instead of an application sending many SQL statements individually, it can call one routine.

5. Encapsulation

The application doesn't necessarily need to know how the operation is implemented internally.

3. Stored Procedure

A **stored procedure** is a named collection of SQL statements stored in the database that is invoked using `CALL`.

Basic idea:

```
CALL procedure_name();
```

A procedure can:

- accept input
- produce output
- modify database data
- return result sets
- use variables
- use conditions
- use loops
- use cursors
- use exception/condition handlers
- perform transactions

MySQL procedures can also use `OUT` and `INOUT` parameters. ([MySQL](#))

4. Basic Procedure Syntax

```
DELIMITER //
```



```
CREATE PROCEDURE procedure_name()  
BEGIN  
    -- SQL statements  
END //
```



```
DELIMITER ;
```

Example:

```
DELIMITER //
```



```
CREATE PROCEDURE GetEmployees()
```

```
BEGIN
    SELECT * FROM employees;
END //

DELIMITER ;
```

Execute:

```
CALL GetEmployees();
```

5. Why Do We Use DELIMITER?

This is extremely important for beginners.

Normally MySQL uses:

```
;
```

as the statement terminator.

Consider:

```
CREATE PROCEDURE GetEmployees()
BEGIN
    SELECT * FROM employees;
END;
```

The MySQL client may interpret the `;` after `SELECT` as the end of the entire `CREATE PROCEDURE` statement.

Therefore, we temporarily change the delimiter:

```
DELIMITER //
```

Now:

```
CREATE PROCEDURE GetEmployees()
BEGIN
```

```
SELECT * FROM employees;  
END //
```

The `;` inside `BEGIN ... END` terminates individual SQL statements, while `//` terminates the entire `CREATE PROCEDURE` statement.

Then restore it:

```
DELIMITER ;
```

Important: `DELIMITER` is primarily a command understood by the MySQL client, not part of the stored routine itself.

6. Procedure Parameters

MySQL procedures support three parameter modes:

Parameter	Meaning
IN	Input
OUT	Output
INOUT	Input + Output

Syntax:

```
CREATE PROCEDURE procedure_name(  
    [IN | OUT | INOUT] parameter_name data_type  
)  
BEGIN  
    ...  
END;
```

If no mode is specified, the parameter is automatically treated as `IN`. ([MySQL](#))

7. IN Parameter

An **IN** parameter sends a value **into** the procedure.

Example:

```
DELIMITER //
```

```
CREATE PROCEDURE GetEmployee(IN emp_id INT)
BEGIN
    SELECT *
    FROM employees
    WHERE employee_id = emp_id;
END //
```

```
DELIMITER ;
```

Call:

```
CALL GetEmployee(101);
```

Conceptually:

```
101
|
v
Procedure
|
v
SELECT employee WHERE id = 101
```

The procedure can modify its local copy of an **IN** parameter, but that modification isn't passed back to the caller. ([MySQL](#))

8. OUT Parameter

An **OUT** parameter sends a value **from the procedure back to the caller**.

Example:

```

DELIMITER //

CREATE PROCEDURE GetEmployeeCount (OUT total INT)
BEGIN
    SELECT COUNT (*)
    INTO total
    FROM employees;
END //

DELIMITER ;

```

Call:

```

CALL GetEmployeeCount (@total);

SELECT @total;

```

Possible output:

```

@total
-----
150

```

Important

An **OUT** parameter starts as **NULL** inside the procedure and receives its value from the procedure.
([MySQL](#))

9. INOUT Parameter

INOUT does both:

```

Caller
|
| initial value
v
Procedure
|

```

```
| modified value
v
Caller
```

Example:

```
DELIMITER //
```

```
CREATE PROCEDURE IncreaseValue(INOUT x INT)
BEGIN
    SET x = x + 10;
END //
```

```
DELIMITER ;
```

Call:

```
SET @value = 50;

CALL IncreaseValue(@value);

SELECT @value;
```

Result:

```
60
```

The caller supplied `50` , and the procedure returned `60` .

10. Procedure Parameter Example Using All Three

```
DELIMITER //
```

```
CREATE PROCEDURE EmployeeExample(
    IN emp_id INT,
    OUT emp_name VARCHAR(100),
```



```
        INOUT bonus DECIMAL(10,2)
    )
BEGIN

    SELECT name
    INTO emp_name
    FROM employees
    WHERE employee_id = emp_id;

    SET bonus = bonus + 1000;

END //

DELIMITER ;
```

Call:

```
SET @name = NULL;
SET @bonus = 5000;

CALL EmployeeExample(101, @name, @bonus);

SELECT @name, @bonus;
```

11. User Variables for OUT/INOUT

When calling a procedure from a normal SQL session, **OUT** and **INOUT** parameters are commonly passed using user-defined variables:

```
SET @result = 0;

CALL MyProcedure(@result);

SELECT @result;
```

MySQL specifically requires a caller variable for an **OUT** or **INOUT** parameter when using **CALL** in this manner. ([MySQL](#))

12. Procedure Returning a Result Set

A procedure can directly return a result set.

Example:

```
DELIMITER //
```

```
CREATE PROCEDURE GetHighSalaryEmployees()  
BEGIN  
    SELECT *  
    FROM employees  
    WHERE salary > 50000;  
END //
```

```
DELIMITER ;
```

Call:

```
CALL GetHighSalaryEmployees();
```

This can return multiple rows and columns.

This is an important difference from stored functions.

13. Stored Function

A **stored function** is a stored routine that **must return a single scalar value**.

It is normally used inside an expression.

For example:

```
SELECT CalculateBonus(50000);
```

or:

```
SELECT name, CalculateBonus(salary)
FROM employees;
```

MySQL describes a stored function as something that can be invoked in an expression and returns a scalar value. ([MySQL](#))

14. Basic Function Syntax

```
DELIMITER //
```



```
CREATE FUNCTION function_name(parameter_name data_type)
RETURNS return_data_type
[characteristics]
BEGIN
    -- statements
    RETURN value;
END //
```



```
DELIMITER ;
```

Example:

```
DELIMITER //
```



```
CREATE FUNCTION CalculateBonus(
    salary DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN salary * 0.10;
END //
```



```
DELIMITER ;
```

Call:

```
SELECT CalculateBonus(50000);
```

Result:

```
5000.00
```

15. Function Parameters

Unlike procedures, MySQL stored functions do **not** support:

```
OUT
INOUT
```

Function parameters are treated as **IN** parameters. ([MySQL](#))

Correct:

```
CREATE FUNCTION CalculateTax(
    salary DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
...
```

Not:

```
CREATE FUNCTION CalculateTax(
    OUT salary DECIMAL(10,2)
)
```

16. RETURN Statement

A stored function must return a value.

Example:

```
RETURN salary * 0.10;
```

The returned value must be compatible with the type specified after **RETURNS** .

```
RETURNS DECIMAL(10,2)
```

therefore the function should return a value appropriate for `DECIMAL(10,2)` .

17. Function Example With IF

```
DELIMITER //
```



```
CREATE FUNCTION GetGrade(mark INT)
RETURNS CHAR(1)
DETERMINISTIC
BEGIN

    IF mark >= 90 THEN
        RETURN 'A';
    ELSEIF mark >= 80 THEN
        RETURN 'B';
    ELSEIF mark >= 70 THEN
        RETURN 'C';
    ELSE
        RETURN 'F';
    END IF;

END //
```



```
DELIMITER ;
```

Use:

```
SELECT GetGrade(85);
```

Result:

```
B
```

18. Function Used With a Table

Suppose:

```
employees
```

```
-----  
employee_id | name | salary  
-----  
1           | John | 50000  
2           | Mary | 60000
```

Function:

```
DELIMITER //  
  
CREATE FUNCTION AnnualSalary(monthly_salary DECIMAL(10,2))  
RETURNS DECIMAL(12,2)  
DETERMINISTIC  
BEGIN  
    RETURN monthly_salary * 12;  
END //  
  
DELIMITER ;
```

Then:

```
SELECT  
    name,  
    salary,  
    AnnualSalary(salary) AS annual_salary  
FROM employees;
```

Result conceptually:

John	50000	600000
Mary	60000	720000

This is one of the major reasons functions are useful.

19. Procedure vs Function – Most Important Difference

Feature	Procedure	Function
Keyword	PROCEDURE	FUNCTION
Invocation	CALL	Used in expressions
Must return value?	No	Yes
RETURN value	No	Yes
IN	Yes	Yes
OUT	Yes	No
INOUT	Yes	No
Can return result set	Yes	No
Can be used in SELECT expression	No	Yes
Can perform transaction statements	Yes, subject to normal transaction rules	No explicit/implicit commit or rollback
Main purpose	Perform an operation	Calculate/return a value
Can have multiple output values	Yes, using OUT / INOUT	No, one scalar return value
Invocation example	CALL p(10);	SELECT f(10);

The result-set and transaction restrictions are specifically documented by MySQL. ([MySQL](#))

20. Simple Way to Remember

Procedure

DO something

```
CALL ProcessOrder(101);
```

Function

GIVE me something

```
SELECT CalculateTax(50000);
```

21. BEGIN ... END

A routine can contain a compound statement:

```
BEGIN

    statement1;
    statement2;
    statement3;

END
```

Example:

```
DELIMITER //
```

```
CREATE PROCEDURE Example()
BEGIN
    DECLARE x INT;

    SET x = 10;

    SELECT x;

END //
```

```
DELIMITER ;
```

BEGIN ... END allows multiple statements and control-flow constructs.

22. Local Variables

Stored routines can declare local variables using `DECLARE` .

Syntax:

```
DECLARE variable_name data_type;
```

Example:

```
DECLARE total DECIMAL(10,2);
```

Assign a value:

```
SET total = 5000;
```

Or:

```
SELECT salary  
INTO total  
FROM employees  
WHERE employee_id = 101;
```

MySQL supports local variables specifically for stored programs, and query results can be assigned using `SELECT ... INTO` .([MySQL](#))

23. Important Rule for DECLARE

`DECLARE` statements must appear at the beginning of their block, before executable statements.

Correct:

```
BEGIN  
    DECLARE x INT;  
    DECLARE y INT;  
  
    SET x = 10;  
    SET y = 20;
```

```
SELECT x + y;  
END
```

Don't put a declaration after executable statements in the same block.

24. Variable Scope

Variables declared inside a block are available within that block and its nested blocks, subject to MySQL's scope rules.

Example:

```
BEGIN  
  
    DECLARE x INT DEFAULT 10;  
  
    BEGIN  
        DECLARE y INT DEFAULT 20;  
  
        SELECT x, y;  
    END;  
  
END
```

`x` is accessible in the inner block, while `y` exists only in the inner block.

25. Assigning Values to Variables

Using SET

```
SET x = 100;
```

Multiple assignments:

```
SET x = 10, y = 20;
```

Using SELECT INTO

```
SELECT salary
INTO v_salary
FROM employees
WHERE employee_id = 101;
```

This is very common in stored procedures/functions.

26. IF Statement

Syntax:

```
IF condition THEN
    statements;
ELSEIF condition THEN
    statements;
ELSE
    statements;
END IF;
```

Example:

```
IF salary >= 50000 THEN
    SET bonus = salary * 0.10;
ELSE
    SET bonus = salary * 0.05;
END IF;
```

27. CASE Statement

Example:

```
CASE
    WHEN salary >= 100000 THEN
        SET category = 'High';
```

```
    WHEN salary >= 50000 THEN
        SET category = 'Medium';
    ELSE
        SET category = 'Low';
END CASE;
```

Or expression-style:

```
CASE department_id
    WHEN 1 THEN SET dept_name = 'HR';
    WHEN 2 THEN SET dept_name = 'IT';
END CASE;
```

28. Loops in Procedures and Functions

MySQL stored programs support several loop structures.

LOOP

```
LOOP
    statements;

    IF condition THEN
        LEAVE loop_name;
    END IF;
END LOOP;
```

WHILE

```
WHILE condition DO
    statements;
END WHILE;
```

REPEAT

```
REPEAT
    statements;
UNTIL condition
END REPEAT;
```

29. WHILE Example

```
DELIMITER //
```



```
CREATE PROCEDURE CountNumbers()
BEGIN
    DECLARE x INT DEFAULT 1;

    WHILE x <= 5 DO
        SELECT x;
        SET x = x + 1;
    END WHILE;

END //
```



```
DELIMITER ;
```

30. REPEAT Example

```
DELIMITER //
```



```
CREATE PROCEDURE CountNumbers()
BEGIN
    DECLARE x INT DEFAULT 1;

    REPEAT
        SELECT x;
        SET x = x + 1;
    UNTIL x > 5
    END REPEAT;
```

```
END //
```

```
DELIMITER ;
```

Difference:

```
WHILE
|
+--> checks condition BEFORE execution
```

```
REPEAT
|
+--> checks condition AFTER execution
```

Therefore, **REPEAT** executes its body at least once.

31. LEAVE and ITERATE

LEAVE

Used to exit a loop/block.

```
LEAVE loop_name;
```

ITERATE

Used to skip to the next iteration.

```
ITERATE loop_name;
```

Example:

```
my_loop: LOOP

    SET x = x + 1;

    IF x = 5 THEN
```

```
        LEAVE my_loop;  
    END IF;
```

```
END LOOP;
```

The label is important when using `LEAVE` / `ITERATE` .

32. Cursors

A cursor allows a stored program to process query results **row by row**.

Conceptually:

```
Query  
|  
v  
Cursor  
|  
+--> Row 1  
+--> Row 2  
+--> Row 3  
+--> ...
```

Typical cursor sequence:

```
DECLARE CURSOR  
    ↓  
DECLARE HANDLER  
    ↓  
OPEN  
    ↓  
FETCH  
    ↓  
process row  
    ↓  
repeat  
    ↓  
CLOSE
```

33. Cursor Syntax

Basic structure:

```
DECLARE done INT DEFAULT FALSE;

DECLARE cur CURSOR FOR
    SELECT employee_id, salary
    FROM employees;

DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET done = TRUE;

OPEN cur;

read_loop: LOOP

    FETCH cur INTO v_id, v_salary;

    IF done THEN
        LEAVE read_loop;
    END IF;

    -- process row

END LOOP;

CLOSE cur;
```

Cursors are generally used inside stored programs when row-by-row processing is genuinely necessary.

34. Condition Handling

Stored programs can handle errors and special conditions using handlers.

General idea:

```
SQL statement
|
```



```
      v
Condition occurs
      |
      v
Handler
      |
      +--> CONTINUE
      |
      +--> EXIT
```

Example:

```
DECLARE CONTINUE HANDLER FOR NOT FOUND
SET done = TRUE;
```

This is particularly common with cursors.

35. CONTINUE vs EXIT Handler

CONTINUE

After handling the condition, execution continues.

```
DECLARE CONTINUE HANDLER
FOR NOT FOUND
SET done = TRUE;
```

EXIT

Execution exits the block where the handler is declared.

```
DECLARE EXIT HANDLER
FOR SQLEXCEPTION
BEGIN
    -- error handling
END;
```

36. SQLSTATE and Conditions

Handlers can respond to conditions such as:

```
NOT FOUND
```

or:

```
SQLEXCEPTION
```

or:

```
SQLWARNING
```

Example:

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK;
END;
```

This is frequently used in transactional procedures.

37. Transactions in Procedures

Procedures can contain transaction statements such as:

```
START TRANSACTION;
COMMIT;
ROLLBACK;
```

Example:

```
DELIMITER //
```



```
CREATE PROCEDURE TransferMoney(
```

```
    IN from_account INT,  
    IN to_account INT,  
    IN amount DECIMAL(10,2)  
)  
BEGIN  
  
    START TRANSACTION;  
  
    UPDATE accounts  
    SET balance = balance - amount  
    WHERE account_id = from_account;  
  
    UPDATE accounts  
    SET balance = balance + amount  
    WHERE account_id = to_account;  
  
    COMMIT;  
  
END //  
  
DELIMITER ;
```

A real implementation should also validate the accounts, amount, balances, affected rows, and error handling.

38. Transactions in Functions

This is an important exam point.

Stored functions **cannot contain statements that perform explicit or implicit commit or rollback.** ([MySQL](#))

Therefore, things like:

```
COMMIT;  
ROLLBACK;  
START TRANSACTION;
```

are not appropriate inside a stored function.

This is one reason procedures are preferred for operations involving multi-step transactions.

39. Result Sets: Procedure vs Function

A procedure can do:

```
SELECT *  
FROM employees;
```

and return the result set to the caller.

A stored function cannot return a result set in this manner.

For example, this is not allowed inside a stored function:

```
SELECT *  
FROM employees;
```

Instead, a function can use:

```
SELECT salary  
INTO v_salary  
FROM employees  
WHERE employee_id = id;
```

and then:

```
RETURN v_salary;
```

MySQL explicitly prohibits statements that return result sets from stored functions. ([MySQL](#))

40. Procedure Can Call a Procedure

Procedures can invoke other procedures.

For example:

```
CALL ValidateEmployee(101);  
CALL CalculateSalary(101);
```

A procedure can therefore be used to organize complex operations into smaller routines.

41. Can a Procedure Call a Function?

Yes.

Example:

```
SET bonus = CalculateBonus(salary);
```

A procedure can use a stored function wherever a function expression is permitted.

42. Can a Function Call a Procedure?

This is an important distinction.

A stored function cannot simply use `CALL` to invoke a procedure in the way a normal client does. Stored functions have significant restrictions because they are invoked as part of an expression.

In practice:

Procedure	→	Function	✓
Procedure	→	Procedure	✓
Function	→	Function	✓
Function	→	Procedure	✗

For exam purposes, remember that **stored functions cannot invoke stored procedures**.

43. DDL Inside Routines

MySQL permits routines to contain certain DDL statements such as:

```
CREATE
DROP
ALTER
```

subject to the applicable restrictions.

However, DDL may have transaction implications because some DDL statements cause implicit commits.

MySQL's routine restrictions should therefore be considered when mixing DDL and transaction logic. ([MySQL](#))

44. **USE** Inside a Routine

You cannot write:

```
USE database_name;
```

inside a stored routine.

When a routine is invoked, MySQL automatically uses the database with which the routine is associated during its execution. ([MySQL](#))

If you need to access a different database, qualify the object:

```
SELECT *
FROM other_database.employees;
```

45. Database Association

A stored procedure/function belongs to a particular database.

You can create one explicitly:

```
CREATE PROCEDURE company.GetEmployees()
...
```

Then invoke it:

```
CALL company.GetEmployees();
```

The routine's associated database matters because MySQL implicitly uses that database while the routine executes. ([MySQL](#))

If the database containing the routine is dropped, its associated stored routines are also dropped. ([MySQL](#))

46. Function Characteristics

A function can specify characteristics such as:

```
DETERMINISTIC
```

or:

```
NOT DETERMINISTIC
```

and data-access characteristics:

```
CONTAINS SQL  
NO SQL  
READS SQL DATA  
MODIFIES SQL DATA
```

The general syntax is:

```
CREATE FUNCTION name (...)  
RETURNS type  
[NOT] DETERMINISTIC  
{CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA}  
...
```

MySQL documents these as routine characteristics. ([MySQL](#))

47. DETERMINISTIC

A deterministic function returns the same result for the same input values, assuming the relevant database/environment doesn't introduce changing factors.

Example:

```
CREATE FUNCTION Square(x INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN x * x;
END;
```

```
Square(5) → 25
Square(5) → 25
Square(5) → 25
```

It doesn't depend on the current time, random number, etc.

48. NOT DETERMINISTIC

A function is non-deterministic if the same inputs can produce different results.

Example:

```
CREATE FUNCTION GetRandomValue()
RETURNS INT
NOT DETERMINISTIC
BEGIN
    RETURN FLOOR(RAND() * 100);
END;
```

Calling it can produce different values.

49. SQL Data Access Characteristics

NO SQL

The routine contains no SQL statements that access SQL data.

CONTAINS SQL

The routine contains SQL statements but doesn't read or modify SQL data.

READS SQL DATA

The routine reads data.

Example:

```
SELECT salary
FROM employees;
```

MODIFIES SQL DATA

The routine modifies data.

Example:

```
UPDATE employees
SET salary = salary + 1000;
```

These characteristics tell MySQL about how the routine interacts with SQL data. ([MySQL](#))

50. SQL SECURITY

MySQL supports:

```
SQL SECURITY DEFINER
```

and:

```
SQL SECURITY INVOKER
```

DEFINER

The routine executes using the privileges of its definer.

INVOKER

The routine executes using the privileges of the user invoking it.

Example:

```
CREATE PROCEDURE GetEmployees()  
SQL SECURITY INVOKER  
BEGIN  
    SELECT * FROM employees;  
END;
```

Security context is important when designing database applications.

MySQL's default security behavior is **DEFINER** unless otherwise specified. ([MySQL](#))

51. DEFINER

A routine can specify:

```
DEFINER = 'user'@'host'
```

For example:

```
CREATE DEFINER = 'admin'@'localhost'  
PROCEDURE GetAccounts()  
BEGIN  
    SELECT * FROM accounts;  
END;
```

If **DEFINER** is omitted, the creator becomes the definer by default. ([MySQL](#))

Be careful when moving routines between development, testing, and production because the referenced definer account must be appropriate.

52. COMMENT Characteristic

You can document a routine:

```
CREATE PROCEDURE GetEmployees()  
COMMENT 'Returns all employees'  
BEGIN  
    SELECT * FROM employees;  
END;
```

The comment can be viewed using metadata commands such as `SHOW CREATE PROCEDURE` .
([MySQL](#))

53. LANGUAGE SQL

MySQL's routine syntax includes:

```
LANGUAGE SQL
```

but MySQL currently supports SQL routines; the `LANGUAGE` characteristic is effectively informational/ignored by the server. ([MySQL](#))

54. Complete Procedure Syntax

The general MySQL syntax is approximately:

```
CREATE  
    [DEFINER = user]  
    PROCEDURE [IF NOT EXISTS] procedure_name  
    ([IN | OUT | INOUT] parameter_name data_type, ...)  
    [characteristics]  
    routine_body;
```

Characteristics can include:

```
COMMENT 'text'  
LANGUAGE SQL  
[NOT] DETERMINISTIC  
CONTAINS SQL  
NO SQL  
READS SQL DATA  
MODIFIES SQL DATA  
SQL SECURITY {DEFINER | INVOKER}
```

This is based on MySQL's documented `CREATE PROCEDURE` syntax. ([MySQL](#))

55. Complete Function Syntax

General form:

```
CREATE  
    [DEFINER = user]  
    FUNCTION [IF NOT EXISTS] function_name  
    (parameter_name data_type, ...)  
    RETURNS return_type  
    [characteristics]  
    routine_body;
```

Function parameters don't have `IN`, `OUT`, or `INOUT` declarations; they are input parameters. ([MySQL](#))

56. Creating a Procedure

Example:

```
DELIMITER //  
  
CREATE PROCEDURE GetEmployeeById(IN p_id INT)  
BEGIN  
  
    SELECT employee_id, name, salary  
    FROM employees  
    WHERE employee_id = p_id;
```

```
END //
```

```
DELIMITER ;
```

57. Calling a Procedure

```
CALL GetEmployeeById(101);
```

If there are no parameters:

```
CALL GetEmployees();
```

The parentheses are used even when the procedure has no parameters. ([MySQL](#))

58. Creating a Function

```
DELIMITER //
```

```
CREATE FUNCTION CalculateAnnualSalary(  
    monthly_salary DECIMAL(10,2)  
)  
RETURNS DECIMAL(12,2)  
DETERMINISTIC  
BEGIN  
    RETURN monthly_salary * 12;  
END //
```

```
DELIMITER ;
```

59. Calling a Function

You don't use `CALL` .

Correct:

```
SELECT CalculateAnnualSalary(50000);
```

You can also use it in expressions:

```
SELECT
    name,
    CalculateAnnualSalary(salary)
FROM employees;
```

Or:

```
SELECT *
FROM employees
WHERE CalculateAnnualSalary(salary) > 600000;
```

60. DROP Procedure

To remove a procedure:

```
DROP PROCEDURE procedure_name;
```

Safer form:

```
DROP PROCEDURE IF EXISTS procedure_name;
```

61. DROP Function

```
DROP FUNCTION function_name;
```

or:

```
DROP FUNCTION IF EXISTS function_name;
```

Be aware that MySQL also has **loadable functions**, so `CREATE FUNCTION` / `DROP FUNCTION` terminology can have another meaning in some contexts. For ordinary SQL programming courses, "stored function" is what you're normally discussing. ([MySQL](#))

62. ALTER PROCEDURE

MySQL provides:

```
ALTER PROCEDURE procedure_name  
...
```

However, `ALTER PROCEDURE` is not a general-purpose way to rewrite the procedure body.

For many changes to the actual routine definition, the usual approach is:

```
DROP PROCEDURE ...
```

followed by:

```
CREATE PROCEDURE ...
```

MySQL documents `ALTER PROCEDURE` as a routine-management statement. ([MySQL](#))

63. ALTER FUNCTION

Similarly:

```
ALTER FUNCTION function_name  
...
```

can alter certain routine characteristics, but it isn't a general "edit the entire function body" command.

For changing the body, recreating the function is commonly used.

64. SHOW CREATE PROCEDURE

To see the definition:

```
SHOW CREATE PROCEDURE GetEmployeeById;
```

It returns the SQL necessary to recreate the procedure. ([MySQL](#))

65. SHOW CREATE FUNCTION

```
SHOW CREATE FUNCTION CalculateAnnualSalary;
```

This displays the stored function definition. ([MySQL](#))

66. SHOW PROCEDURE STATUS

```
SHOW PROCEDURE STATUS;
```

You can filter:

```
SHOW PROCEDURE STATUS  
WHERE Db = 'company';
```

Similarly:

```
SHOW FUNCTION STATUS;
```

MySQL provides these statements for routine metadata. ([MySQL](#))

67. INFORMATION_SCHEMA

You can inspect routines using:

```
INFORMATION_SCHEMA.ROUTINES
```

For example:

```
SELECT
    ROUTINE_SCHEMA,
    ROUTINE_NAME,
    ROUTINE_TYPE
FROM INFORMATION_SCHEMA.ROUTINES
WHERE ROUTINE_SCHEMA = 'company';
```

This can show whether the routine is:

```
PROCEDURE
```

or:

```
FUNCTION
```

MySQL specifically recommends `INFORMATION_SCHEMA.ROUTINES` for routine metadata. ([MySQL](#))

68. INFORMATION_SCHEMA.PARAMETERS

Parameter information can be obtained from:

```
INFORMATION_SCHEMA.PARAMETERS
```

For example:

```
SELECT
    SPECIFIC_NAME,
    PARAMETER_NAME,
    PARAMETER_MODE,
    DATA_TYPE
```

```
FROM INFORMATION_SCHEMA.PARAMETERS
WHERE SPECIFIC_SCHEMA = 'company';
```

MySQL exposes routine parameter metadata through this table. ([MySQL](#))

69. Privileges

Creating a stored procedure/function requires the appropriate routine privileges.

For `CREATE PROCEDURE` and `CREATE FUNCTION`, MySQL documents the `CREATE ROUTINE` privilege requirement. ([MySQL](#))

Routine creators are also automatically granted `ALTER ROUTINE` and `EXECUTE` by default, unless the `automatic_sp_privileges` system variable is disabled. ([MySQL](#))

70. Procedure and Function Naming

Use descriptive names.

Good:

```
GetEmployeeSalary
CalculateTax
UpdateEmployeeSalary
GetDepartmentCount
```

Bad:

```
p1
f1
abc
test
```

Also avoid unnecessarily using the names of built-in MySQL functions. MySQL has special name-resolution rules for such cases. ([MySQL](#))

71. Parameter Naming Convention

A common convention is to prefix parameters:

p_

Examples:

```
p_employee_id  
p_salary  
p_department_id
```

And local variables:

v_

Examples:

```
v_salary  
v_total  
v_count
```

This isn't mandatory, but it makes code much easier to understand.

Example:

```
CREATE PROCEDURE GetEmployee(  
    IN p_employee_id INT  
)  
BEGIN  
  
    DECLARE v_salary DECIMAL(10,2);  
  
    SELECT salary  
    INTO v_salary  
    FROM employees  
    WHERE employee_id = p_employee_id;  
  
    SELECT v_salary;
```

```
END;
```

This also reduces confusion between column names, parameters, and local variables.

72. Parameter vs Local Variable

Parameter

Comes from or communicates with the caller:

```
IN p_id INT
```

Local variable

Exists only while the routine executes:

```
DECLARE v_salary DECIMAL(10,2);
```

Conceptually:

```
Caller
|
| parameter
v
Routine
|
| local variables
v
Processing
|
| OUT/INOUT/result
v
Caller
```

73. Important Name Resolution Issue

Suppose you write:

```
CREATE PROCEDURE GetEmployee(IN id INT)
BEGIN
    SELECT *
    FROM employees
    WHERE id = id;
END;
```

This can be problematic because `id` can be ambiguous.

Prefer:

```
CREATE PROCEDURE GetEmployee(IN p_id INT)
BEGIN
    SELECT *
    FROM employees
    WHERE employee_id = p_id;
END;
```

Using clear prefixes like `p_` and `v_` is therefore good practice.

74. SELECT INTO

Very important for functions and procedures.

Example:

```
DECLARE v_salary DECIMAL(10,2);

SELECT salary
INTO v_salary
FROM employees
WHERE employee_id = p_id;
```

Then:

```
SELECT v_salary;
```

or:

```
RETURN v_salary;
```

for a function.

75. Procedure Example – Complete

```
DELIMITER //
```

```
CREATE PROCEDURE GetEmployeeDetails(  
    IN p_employee_id INT  
)  
BEGIN  
  
    DECLARE v_salary DECIMAL(10,2);  
  
    SELECT salary  
    INTO v_salary  
    FROM employees  
    WHERE employee_id = p_employee_id;  
  
    SELECT  
        employee_id,  
        name,  
        salary,  
        v_salary * 12 AS annual_salary  
    FROM employees  
    WHERE employee_id = p_employee_id;  
  
END //
```

```
DELIMITER ;
```

Call:

```
CALL GetEmployeeDetails(101);
```

76. Function Example – Complete

```
DELIMITER //
```

```
CREATE FUNCTION GetAnnualSalary(  
    p_monthly_salary DECIMAL(10,2)  
)  
RETURNS DECIMAL(12,2)  
DETERMINISTIC  
BEGIN  
  
    DECLARE v_annual_salary DECIMAL(12,2);  
  
    SET v_annual_salary = p_monthly_salary * 12;  
  
    RETURN v_annual_salary;  
  
END //
```

```
DELIMITER ;
```

Use:

```
SELECT GetAnnualSalary(50000);
```

77. Function That Reads a Table

Functions can read data, subject to the restrictions applicable to stored functions.

Example:

```
DELIMITER //
```

```
CREATE FUNCTION GetEmployeeSalary(  
    p_employee_id INT  
)  
RETURNS DECIMAL(10,2)  
READS SQL DATA
```

```
BEGIN
```

```
DECLARE v_salary DECIMAL(10,2);

SELECT salary
INTO v_salary
FROM employees
WHERE employee_id = p_employee_id;

RETURN v_salary;
```

```
END //
```

```
DELIMITER ;
```

Use:

```
SELECT GetEmployeeSalary(101);
```

78. Procedure for Updating Data

```
DELIMITER //
```

```
CREATE PROCEDURE IncreaseSalary(
    IN p_employee_id INT,
    IN p_amount DECIMAL(10,2)
)
MODIFIES SQL DATA
BEGIN

    UPDATE employees
    SET salary = salary + p_amount
    WHERE employee_id = p_employee_id;

END //
```

```
DELIMITER ;
```

Call:


```
CALL IncreaseSalary(101, 5000);
```

79. Function vs Procedure – Practical Decision

Ask:

"Am I performing an operation?"

Use a **procedure**.

Examples:

```
Transfer money  
Insert an order  
Update salary  
Delete an employee  
Process payroll  
Generate a report
```

"Am I calculating/returning a value?"

Use a **function**.

Examples:

```
Calculate tax  
Calculate bonus  
Convert temperature  
Calculate age  
Calculate annual salary  
Determine grade
```

80. Important Restrictions on Stored Functions

This is one of the most important areas for exams.

A stored function has stricter restrictions than a procedure.

Functions cannot:

- return a result set directly
- use `CALL` to invoke a procedure in the normal way
- use explicit transaction control such as `COMMIT` / `ROLLBACK`
- use `OUT` or `INOUT` parameters
- behave like procedures that simply return arbitrary result sets

MySQL's documented restrictions explain the result-set and transaction limitations. ([MySQL](#))

81. Restrictions on Stored Routines

Stored programs have additional restrictions concerning certain statements, prepared statements, dynamic SQL, transactions, and other database operations.

For example, routine parameters cannot be directly referenced inside prepared statements created within the routine. ([MySQL](#))

For an exam, the most important restrictions to remember are:

```
Function
|
+--> no OUT/INOUT
+--> no result set
+--> no transaction COMMIT/ROLLBACK
+--> must return a value
```

82. Prepared Statements / Dynamic SQL

MySQL stored programs have restrictions around prepared statements.

Routine parameters cannot be directly referenced in statements prepared within the routine. ([MySQL](#))

Dynamic SQL is therefore an area where you need to pay attention to MySQL's stored-program restrictions rather than assuming that every normal SQL statement can be dynamically constructed inside a function/procedure.

83. Recursion

Procedures

Stored procedures can be recursive, but recursion is disabled by default.

MySQL uses:

```
max_sp_recursion_depth
```

to control stored procedure recursion. A value greater than zero enables recursion to the specified depth. ([MySQL](#))

Functions

MySQL stored functions **cannot be recursive**. ([MySQL](#))

So:

```
Procedure recursion → possible  
Function recursion  → not allowed
```

84. Example of Procedure Recursion

Conceptually:

```
CALL FactorialProcedure(5, @result);
```

The procedure can call itself, subject to recursion configuration.

However, for mathematical calculations such as factorial, a stored function would normally be more natural—but MySQL's restriction means you should not implement recursive stored functions.

85. SQL Mode and Stored Routines

An important advanced point:

MySQL stores the `sql_mode` that was active when a routine was created or altered.

The routine executes using that stored SQL mode when invoked. ([MySQL](#))

Therefore, changing the session's SQL mode later doesn't necessarily mean the routine will execute using that new mode.

This can matter when moving routines between environments.

86. Stored Routine Security

There are two important concepts:

```
DEFINER
  ↓
Who defines/security context

INVOKER
  ↓
Who calls/security context
```

Example:

```
SQL SECURITY DEFINER
```

means the routine uses the definer's privileges for its internal operations.

Whereas:

```
SQL SECURITY INVOKER
```

uses the invoker's privileges.

This is particularly important when a user has:

```
EXECUTE privilege
```

on a routine but doesn't have direct privileges on the underlying tables.

87. Advantages of Stored Procedures

1. Reusability

Write once, call many times.

2. Security

Can restrict direct table access and expose controlled operations.

3. Centralized business logic

Logic resides in the database rather than being duplicated across applications.

4. Reduced client-server communication

One call can perform multiple operations.

5. Can return result sets

Useful for reports and queries.

6. Supports transactions

Very useful for multi-step database operations.

7. Supports complex logic

Can use:

- IF
 - CASE
 - loops
 - cursors
 - handlers
 - variables
-

88. Disadvantages of Stored Procedures

1. Database-specific code

MySQL stored procedure syntax isn't necessarily portable to PostgreSQL, SQL Server, Oracle, etc.

2. More logic on database server

Heavy routine usage can increase database-server workload.

3. Debugging can be harder

Application-language debugging tools are often better than database routine debugging.

4. Version control can require extra discipline

Stored routines must be properly included in database migration/deployment systems.

5. Can become difficult to maintain

Very large procedures can become complicated.

6. Logic may become tightly coupled to the database

Moving business logic between database systems can become difficult.

MySQL itself notes the tradeoff: stored routines can reduce information sent between client and server, but this can increase server-side load. ([MySQL](#))

89. Advantages of Functions

Functions are particularly useful for:

- calculations
- transformations
- reusable expressions
- derived values
- formatting/conversion logic
- encapsulating frequently used calculations

Example:

```
SELECT
    name,
    salary,
    CalculateTax(salary)
FROM employees;
```

This is much cleaner than repeating a complicated expression everywhere.

90. Disadvantages of Functions

Because functions can be invoked inside SQL expressions, they have stricter restrictions.

They can also cause performance problems if used carelessly.

For example:

```
SELECT CalculateSomething(salary)
FROM millions_of_employees;
```

may execute the function many times.

If the function performs database queries internally, the cost can become significant.

So don't automatically use a function simply because it makes SQL look cleaner.

91. Procedure vs Function vs Trigger

Don't confuse these three.

Feature	Procedure	Function	Trigger
Explicitly called?	Yes, <code>CALL</code>	Yes, as expression	No
Has parameters?	Yes	Yes	No normal parameters
Must return value?	No	Yes	No
Runs automatically?	No	No	Yes
Associated with table event?	No	No	Yes
Can return result set?	Yes	No	No
Typical purpose	Perform operation	Calculate value	Automatically react to table event

92. Procedure vs Function vs Trigger Example

Procedure

```
CALL UpdateEmployeeSalary(101, 5000);
```

Function

```
SELECT CalculateTax(50000);
```

Trigger

```
INSERT into employees
    ↓
    Trigger fires
    ↓
    Insert audit record
```

The trigger is automatically activated by a database event.

93. Procedure vs Function vs View

Another common exam comparison:

Feature	Procedure	Function	View
Stored database object	Yes	Yes	Yes
Parameters	Yes	Yes	No normal parameters
Called using	CALL	Expression	SELECT
Must return value	No	Yes	Represents query result
Contains procedural logic	Yes	Yes	No procedural code
Can have loops	Yes	Yes	No
Can modify data	Yes	Restricted	Normally no direct DML through arbitrary view logic
Main purpose	Operations	Calculations	Virtual table/query

94. Multiple Outputs From Procedure

Suppose you need:

```
Employee name  
Employee salary  
Department
```

A procedure can use multiple `OUT` parameters:

```
CREATE PROCEDURE GetEmployeeInfo(  
    IN p_id INT,  
    OUT p_name VARCHAR(100),  
    OUT p_salary DECIMAL(10,2),  
    OUT p_department VARCHAR(100)  
)  
...
```

A function cannot directly return three independent scalar values.

You could return a single structured value in certain designs, but that is fundamentally different from having multiple **OUT** parameters.

For standard MySQL exam questions:

```
Multiple output values → Procedure  
One return value      → Function
```

95. Procedure Returning a Result Set vs OUT Parameter

A procedure has two common ways of communicating results:

Result set

```
SELECT name, salary  
FROM employees;
```

OUT parameters

```
OUT p_name VARCHAR(100),  
OUT p_salary DECIMAL(10,2)
```

So:

```
Procedure  
|  
+--> Result set  
|  
+--> OUT parameters  
|  
+--> INOUT parameters
```

A function fundamentally returns its result through:

```
RETURN value;
```

96. Error Handling Example

A more realistic procedure might be:

```
DELIMITER //
```



```
CREATE PROCEDURE TransferMoney(  
    IN p_from INT,  
    IN p_to INT,  
    IN p_amount DECIMAL(10,2)  
)  
BEGIN  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    START TRANSACTION;  
  
    UPDATE accounts  
    SET balance = balance - p_amount  
    WHERE account_id = p_from;  
  
    UPDATE accounts  
    SET balance = balance + p_amount  
    WHERE account_id = p_to;  
  
    COMMIT;  
  
END //
```



```
DELIMITER ;
```

This demonstrates the combination of:

- parameters

- transaction
- handler
- rollback
- update
- commit
- error propagation

`RESIGNAL` is useful when you want to handle a condition and then propagate the error.

97. SIGNAL

You can deliberately generate an error using `SIGNAL`.

Example:

```
IF p_amount <= 0 THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Amount must be greater than zero';
END IF;
```

This is useful for validation.

Conceptually:

```
Invalid input
    ↓
SIGNAL
    ↓
Error returned to caller
```

98. Function Validation Example

```
DELIMITER //
```



```
CREATE FUNCTION CalculateDiscount(
    p_price DECIMAL(10,2)
)
```

```
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN

    IF p_price < 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Price cannot be negative';
    END IF;

    RETURN p_price * 0.10;

END //
```

DELIMITER ;

99. Important Difference: **SELECT** in Procedure vs Function

Procedure

Allowed:

```
SELECT name, salary
FROM employees;
```

because it can return a result set.

Function

Not allowed as a result-set-producing statement.

Instead:

```
SELECT salary
INTO v_salary
FROM employees
WHERE employee_id = p_id;
```

```
RETURN v_salary;
```

This distinction is extremely likely to appear in exams. ([MySQL](#))

100. Function Invocation Examples

A stored function can be used in many expressions.

SELECT

```
SELECT CalculateTax(50000);
```

SELECT column

```
SELECT name, CalculateTax(salary)
FROM employees;
```

WHERE

```
SELECT *
FROM employees
WHERE CalculateTax(salary) > 5000;
```

ORDER BY

```
SELECT name, CalculateTax(salary) AS tax
FROM employees
ORDER BY tax DESC;
```

Stored functions are expressions, which is why they can be integrated into SQL statements. ([MySQL](#))

101. Procedure Invocation

A procedure is invoked using:

```
CALL procedure_name(arguments);
```

Example:

```
CALL IncreaseSalary(101, 5000);
```

A procedure itself isn't normally an expression.

You don't write:

```
SELECT IncreaseSalary(101, 5000);
```

102. Routine Namespace

Procedure and function names are stored as database objects and are associated with a database.

Be careful with naming collisions, particularly with built-in functions.

MySQL notes that if a stored routine has the same name as a built-in SQL function, special parsing rules can apply; avoiding such names is recommended. ([MySQL](#))

103. Stored Routine Lifecycle

Think of a routine as having this lifecycle:

```
CREATE
  ↓
STORE in database
  ↓
EXECUTE
  ↓
ALTER / RECREATE
  ↓
DROP
```

Common commands:

```
CREATE PROCEDURE ...
```

```
CREATE FUNCTION ...
```

```
CALL ...
```

```
ALTER PROCEDURE ...
```

```
ALTER FUNCTION ...
```

```
SHOW CREATE PROCEDURE ...
```

```
SHOW CREATE FUNCTION ...
```

```
DROP PROCEDURE ...
```

```
DROP FUNCTION ...
```

MySQL provides these commands for routine management. ([MySQL](#))

104. Where Are Stored Routines Stored?

MySQL stores routine information in its data dictionary.

You normally inspect it through:

```
INFORMATION_SCHEMA.ROUTINES
```

and:

```
INFORMATION_SCHEMA.PARAMETERS
```

rather than directly manipulating internal dictionary tables. ([MySQL](#))

105. Stored Procedure Complete Example

Suppose we have:


```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    salary DECIMAL(10,2)  
);
```

Procedure:

```
DELIMITER //
```

```
CREATE PROCEDURE EmployeeSalaryInfo(  
    IN p_employee_id INT,  
    OUT p_annual_salary DECIMAL(12,2)  
)  
BEGIN  
  
    DECLARE v_monthly_salary DECIMAL(10,2);  
  
    SELECT salary  
    INTO v_monthly_salary  
    FROM employees  
    WHERE employee_id = p_employee_id;  
  
    SET p_annual_salary = v_monthly_salary * 12;  
  
    SELECT  
        employee_id,  
        name,  
        salary,  
        p_annual_salary AS annual_salary  
    FROM employees  
    WHERE employee_id = p_employee_id;  
  
END //
```

```
DELIMITER ;
```

Call:

```
SET @annual = 0;  
  
CALL EmployeeSalaryInfo(101, @annual);
```

```
SELECT @annual;
```

This demonstrates:

- `IN`
 - `OUT`
 - local variable
 - `SELECT INTO`
 - `SET`
 - result set
 - `CALL`
-

106. Stored Function Complete Example

```
DELIMITER //
```

```
CREATE FUNCTION EmployeeAnnualSalary(  
    p_employee_id INT  
)  
RETURNS DECIMAL(12,2)  
READS SQL DATA  
BEGIN  
  
    DECLARE v_salary DECIMAL(10,2);  
  
    SELECT salary  
    INTO v_salary  
    FROM employees  
    WHERE employee_id = p_employee_id;  
  
    RETURN v_salary * 12;  
  
END //
```

```
DELIMITER ;
```

Call:

```
SELECT EmployeeAnnualSalary(101);
```

Or:

```
SELECT
    employee_id,
    name,
    EmployeeAnnualSalary(employee_id) AS annual_salary
FROM employees;
```

107. Performance Considerations

Stored routines aren't automatically faster.

Potential benefits:

```
Less client ↔ server communication
Centralized processing
Reusable execution plan/logic benefits in some situations
```

Potential costs:

```
More database-server work
Repeated function calls
Row-by-row cursor processing
Complex nested routines
```

In particular, avoid unnecessary cursor-based processing when a single SQL statement can perform the operation.

For example, prefer:

```
UPDATE employees
SET salary = salary * 1.10
WHERE department_id = 5;
```

over processing every employee individually with a cursor when the operation is naturally set-based.

108. Set-Based SQL vs Cursor

Cursor approach

```
Get row 1
Update row 1
Get row 2
Update row 2
Get row 3
Update row 3
...
```

Set-based approach

```
UPDATE employees
SET salary = salary * 1.10
WHERE department_id = 5;
```

The second approach is usually preferable when possible.

109. Best Practices

1. Use meaningful names

```
GetEmployeeSalary
```

instead of:

```
p1
```

2. Prefix parameters

```
p_employee_id
```

3. Prefix local variables

```
v_salary
```

4. Keep routines focused

A routine should ideally have one clear responsibility.

5. Validate input

```
IF p_amount <= 0 THEN  
    SIGNAL ...  
END IF;
```

6. Handle errors

Use appropriate handlers.

7. Use transactions where appropriate

Especially for multi-step operations.

8. Prefer set-based operations

Don't use cursors when ordinary SQL can solve the problem.

9. Be careful with functions in large queries

A function may be executed many times.

10. Document routines

Use comments and clear names.

11. Manage privileges carefully

Don't give more permissions than necessary.

12. Include routines in database migrations/version control

Don't manually maintain production routines without tracking their definitions.

110. Common Errors

Error 1: Forgetting **DELIMITER**

Problem:

```
CREATE PROCEDURE test()  
BEGIN  
    SELECT 1;  
END;
```

Depending on the client, the internal **;** can cause parsing problems.

Use:

```
DELIMITER //  
  
CREATE PROCEDURE test()  
BEGIN  
    SELECT 1;  
END //  
  
DELIMITER ;
```

Error 2: Forgetting parentheses

Wrong:

```
CREATE PROCEDURE test
```

Correct:

```
CREATE PROCEDURE test()
```

MySQL requires the parameter list, even when empty. ([MySQL](#))

Error 3: Calling a procedure like a function

Wrong:

```
SELECT GetEmployees();
```

Correct:

```
CALL GetEmployees();
```

Error 4: Calling a function using CALL

Wrong:

```
CALL CalculateTax(50000);
```

Normally:

```
SELECT CalculateTax(50000);
```

Error 5: Using OUT in a function

Wrong:

```
CREATE FUNCTION f(OUT x INT)
```

Stored function parameters are input parameters. ([MySQL](#))

Error 6: Forgetting RETURN

Wrong:

```
CREATE FUNCTION f(x INT)
RETURNS INT
BEGIN
    SET x = x * 2;
END;
```

Correct:

```
CREATE FUNCTION f(x INT)
RETURNS INT
BEGIN
    RETURN x * 2;
END;
```

Error 7: Returning a result set from a function

Wrong conceptually:

```
SELECT *
FROM employees;
```

inside a stored function.

Instead:

```
SELECT salary
INTO v_salary
FROM employees
WHERE employee_id = p_id;

RETURN v_salary;
```

11. Exam-Focused Differences

If you have to write a **5-mark answer**, remember this table:

Point	Procedure	Function
Definition	Stored program for performing operations	Stored program for returning a value
Invocation	CALL	Expression
Return	Not mandatory	Mandatory
IN	Yes	Yes
OUT	Yes	No
INOUT	Yes	No
Result set	Yes	No
Transaction control	Can use transaction statements	Cannot use explicit/implicit commit/rollback
Recursion	Possible	Not allowed
Typical use	Insert/update/delete/process/report	Calculation/value transformation

112. One-Minute Revision

Memorize this:

```

STORED ROUTINE
|
+-----+
|               |
PROCEDURE       FUNCTION
|               |
CALL            SELECT/expression
|               |
Doesn't need    MUST return
to return value one value
|               |
IN / OUT / INOUT
|               |
Can return result set
|               |
Can perform transaction control

```

113. Most Important Syntax to Memorize

Procedure

```
DELIMITER //
```

```
CREATE PROCEDURE procedure_name(  
    IN p_input INT,  
    OUT p_output INT  
)  
BEGIN  
  
    -- statements  
  
    SET p_output = ...;  
  
END //
```

```
DELIMITER ;
```

Call:

```
CALL procedure_name(value, @output);  
  
SELECT @output;
```

Function

```
DELIMITER //
```

```
CREATE FUNCTION function_name(  
    p_input INT  
)  
RETURNS INT  
DETERMINISTIC  
BEGIN  
  
    -- statements
```

```
RETURN value;
```

```
END //
```

```
DELIMITER ;
```

Call:

```
SELECT function_name(value);
```

114. Final Mental Model

Think of a **procedure as a command**:

```
"Database, DO this."
```

```
CALL TransferMoney(...);  
CALL GenerateReport(...);  
CALL UpdateSalary(...);
```

Think of a **function as a calculator**:

```
"Database, GIVE me this value."
```

```
SELECT CalculateTax(...);  
SELECT CalculateBonus(...);  
SELECT GetAnnualSalary(...);
```

That's the fundamental distinction from which almost everything else follows.

High-Yield Facts to Memorize for an Exam

1. **Stored routine = procedure or function.**
2. Procedure is invoked with **CALL**.
3. Function is invoked **inside an expression**.
4. A function **must return a value**.
5. A procedure doesn't have to return a value.

6. Procedure supports **IN** , **OUT** , **INOUT** .
7. Function parameters are effectively **IN only**.
8. Procedure can return **result sets**.
9. Stored function cannot directly return a result set.
10. Functions cannot perform transaction **COMMIT** / **ROLLBACK** .
11. Procedures can perform transaction control, subject to normal MySQL transaction rules.
12. Functions cannot be recursive.
13. Stored procedures can be recursive, but recursion is disabled by default and controlled by `max_sp_recursion_depth` . ([MySQL](#))
14. **DELIMITER** is used by the MySQL client when defining multi-statement routines.
15. **BEGIN ... END** creates a compound statement.
16. **DECLARE** is used for local variables, cursors, and handlers.
17. **SELECT ... INTO** stores query results in variables.
18. **IF** , **CASE** , **LOOP** , **WHILE** , and **REPEAT** provide control flow.
19. **DECLARE HANDLER** handles conditions/errors.
20. **SIGNAL** can deliberately raise an error.
21. **DEFINER** and **INVOKER** control the routine's security context.
22. **DETERMINISTIC** describes whether the same inputs are expected to produce the same result.
23. **READS SQL DATA** means the routine reads SQL data.
24. **MODIFIES SQL DATA** means the routine modifies SQL data.
25. **USE database** is not permitted inside a stored routine.
26. Routine definitions can be inspected using **SHOW CREATE PROCEDURE/FUNCTION** .
27. Routine metadata is available through **INFORMATION_SCHEMA.ROUTINES** .
28. Parameter metadata is available through **INFORMATION_SCHEMA.PARAMETERS** .
29. **DROP PROCEDURE** removes a procedure.
30. **DROP FUNCTION** removes a function. ([MySQL](#))

For the official reference, the most useful MySQL pages are the [stored routine syntax documentation](#) and [CREATE PROCEDURE/CREATE FUNCTION documentation](#). ([MySQL](#))